Control Theory.

1. Dynamic : ① State vector $x(t) = \begin{bmatrix} \\ \\ \\ \\ \end{bmatrix}$ (contains all information needed to predict system evolution with known future input).

② $u(t)$: controller ~~output~~ input. (what controller commands)

③. $f(x,u)$: dynamic function (the physic of a system)
(with given current state and ~~future~~ input. how will state begin to change:)

④ $y(t)$: measured output (sensors).

⑤ $h(x,u)$: measurement function (how internal state produce sensor readings)

~~dyn~~ · deterministic dynamic system (: non-linear) :

$$\dot{x}(t) = f(x(t), u(t)) \ , \ y(t) = h(x(t), u(t)).$$

$$\begin{cases} \dot{x}(t) = f(x(t), u(t)) \rightarrow \text{state change depends on current state and input ruled by physics.} \\ y(t) = h(x(t), u(t)). \rightarrow \text{measuremene depends on current state and input.} \end{cases}$$

↓

discrete

$$\begin{cases} x_{k+1} = f_d(x_k, u_k) \\ y_k = h_d(x_k, u_k) \end{cases}$$

PID Control: reference ↓

① error: $e(t) = r(t) - y(t)$.

$$u(t) = k_p e(t) + k_i \int_0^t e(\tau)\,d\tau + k_d \frac{d(e(t))}{dt}.$$

② Apply Laplace transform: (0 initial).

$$\mathcal{L}(k_p e(t)) = K_p E(s).$$

$$\because I(t) = k_i \int_0^t e(\tau)\,d\tau \qquad k_i \cdot E(s)$$

$$\therefore \frac{dI(t)}{dt} = k_i \cdot e(t). \rightarrow \mathcal{L}\{\dot{I}(t)\} = sI(s) - I(0) = sI(s)$$

$$sI(s) = k_i E(s) \rightarrow I(s) = k_i \frac{E(s)}{s}$$

$$\mathcal{L}\left\{k_d \frac{de(t)}{dt}\right\} = k_d \cdot s \cdot E(s) - e(0) = k_d s E(s)$$

$$\therefore \mathcal{L}\{u(t)\} = \left(k_p + \frac{k_i}{s} + k_d s\right) \cdot E(s) = U(s)$$

$$\text{transfer function} = C(s) = \frac{U(s)}{E(s)} = \frac{\text{output}}{\text{input}} = k_p + \frac{k_i}{s} + k_d s.$$

$$= \frac{k_p s + k_i + k_d s^2}{s}$$

suppose plant (system) has a transfer function:

$$G(s) = \frac{Y(s)}{U(s)} = \frac{\text{measured output}}{\text{input}}.$$

then $E(s) = R(s) - Y(s)$. since $U(s) = C(s) \cdot E(s)$.

then $Y(s) = G(s) \cdot U(s)$

$$Y(s) = G(s) \cdot C(s) \cdot E(s) = G(s) \cdot C(s) \cdot (R(s) - Y(s)).$$

so $(1 + G(s)C(s))Y(s) = G(s)C(s)R(s)$.

measured output → $\frac{Y(s)}{R(s)} = \frac{C(s)G(s)}{1 + C(s)G(s)} = T(s)$

reference. →

example :

PID : $J\ddot{q} + b\dot{q} + kq = \tau$. (if initials $q(0)=0$, $\dot{q}(0)=0$.)

joine :

$$\mathcal{L}\{\tau(t)\} = J\cdot s^2\cdot Q(s) + b\cdot s\cdot Q(s) + kQ(s) = \tau(s)$$

$$G(s)=\frac{Q(s)}{\tau(s)} = \frac{1}{Js^2+bs+k}$$

PID : $C(s) =$ $\frac{k_d s^2 + k_p s + k_i}{s}$

$$\frac{Y(s)}{R(s)} = \frac{GC}{1+CG} \rightarrow \text{let } 1+CG=0.$$

$$1+GC = \frac{(k_d s^2 + k_p s + k_i)}{s(Js^2+bs+k)} + 1 = 0.$$

$$(k_d s^2 + k_p s + k_i) + s(Js^2 + bs^2 + ks = 0.$$

$$\boxed{Js^3 + s^2(k_d + b) + s(k_p + k) + k_i = 0.}$$

$$\boxed{s^3 + \frac{s^2(k_d+b)}{J} + s\left(\frac{k_p+k}{J}\right) + \frac{k_i}{J} = 0.}$$

mass spring damper: $m\ddot{x} + c\dot{x} + kx = 0.$

let $x(t) = Ae^{st}$. $\dot{x}(t) = sAe^{st}$, $\ddot{x}(t) = s^2Ae^{st}$.

$$(ms^2 + cs + k)Ae^{st} = 0.$$

$$ms^2 + cs + k = 0 \rightarrow s^2 + \frac{c}{m}s + \frac{k}{m} = 0.$$

if no damp: $s^2 + \frac{k}{m} = 0 \rightarrow s = \pm\sqrt{\frac{k}{m}}\,j$

$\omega_n = \sqrt{\frac{k}{m}}$ (natural frequency).

Critical damping: no imaginary roots from $(ms^2 + cs + k)$.

since $s = \frac{-c \pm \sqrt{c^2 - 4mk}}{2m}$ $\rightarrow$ $c^2 - 4mk \geq 0$

$c^2 \geq 4mk$

critical damped. $\boxed{c \geq 2\sqrt{2mk}}$

damping ratio $= \delta = \frac{c}{c_{critical}} = \frac{c}{2\sqrt{km}}$

$$2\omega_n \cdot \delta = 2 \cdot \left(\frac{c}{2\sqrt{km}}\right) \cdot \left(\sqrt{\frac{k}{m}}\right) = c \cdot \sqrt{\frac{k}{m} \cdot \frac{1}{km}} = \frac{c}{m}.$$

so $\frac{c}{m} = 2\omega_n \cdot \delta$, $\frac{k}{m} = \omega_n^2$.

so $s^2 + 2\omega_n \cdot \delta s + \omega_n^2 = 0.$

($2\omega_n$: natural frequency; δ: damping ratio; ω_n^2)

since close-loop pid introduce one more pole.

$$\text{so let} \begin{cases} (s+p)\cdot(s^2 + 2\delta\omega_n s + \omega_n^2) = D(s) \rightarrow \text{from physic} \\ s^3 + s^2\cdot\frac{kd+b}{J} + s\left(\frac{kp+k}{J}\right) + \frac{ki}{J} = D(s) \rightarrow \text{from PID.} \end{cases}$$

$$s^3 + 2\delta\omega_n s^2 + \omega_n^2 s + ps^2 + 2\delta\omega_n sp + \omega_n^2 p$$

$$= \begin{cases} s^3 + s^2(2\omega_n\delta + p) + s(2\delta\omega_n p + \omega_n^2) + \omega_n^2 p \\ s^3 + s^2\left(\frac{kd+b}{J}\right) + s\left(\frac{kp+k}{J}\right) + \frac{ki}{J} \end{cases}$$

$$\text{so} \begin{cases} kd = J(2\omega_n\delta + p) - b \\ kp = J(2\delta\omega_n p + \omega_n^2) - k \\ ki = J\cdot kp\cdot\omega_n^2 \end{cases} \rightarrow \text{determine system physical behavior by tuning controller coefficients.}$$

LQR (Linear Quadratic Regulator).

Problem & objective:

assume a linear model and penalizes state deviation and control effort quadratically.

① discrete dynamics: $x_{k+1} = Ax_k + Bu_k$.

x_{k+1} ↓ next state; Ax_k ↓ current state; Bu_k ↓ input comand.

state: $x_k = x_{k\,actual} - x_{k\,desired}$.

define: Q: state error penalty wefficient. (immediate)

R: system effort penalty coefficient.

then $x_k^T \cdot Q \cdot x_k = \sum_{i=1}^{rank\,of\,x} q_i e_{qi}^2$ (just vector square)

$u_k^T R u_k$ = vector square penalty.

if the controller input goes as: $u_0, u_1, \cdots u_{N-1}$,

then final input = u_{N-1}, final state = $x_k = Ax_{N-1} + Bu_{N-1}$

so define cose function:

$$J = x_N^T \cdot Q x_N + \sum_{k=0}^{N-1} \left(x_k^T Q x_k + u_k^T R u_k \right)$$

↓ final state cost. ↓ (state + controller) effort cost during control.

objective of LQR: min(J).

since $x_{k+1} = Ax_k + Bu_k$.

$$x_N = Ax_{N-1} + Bu_{N-1}.$$

$$x_N^T = (Ax_{N-1} + Bu_{N-1})^T$$

so

$$J = (Ax_{N-1} + Bu_{N-1})^T \cdot Q \cdot (Ax_{N-1} + Bu_{N-1}) + \sum_{k=0}^{N-1} (x_k^T Q x_k + u_k^T R u_k).$$

---

define cost to go:

$$V_{k+1}(x) = x_{k+1}^T P_{k+1} \cdot x_{k+1}$$

↓ all future optimal cost.

so if current = k, cost to go = (k+1):

$$V_k(x_k) = \min \left(x_N^T Q x_N + \sum_{i=k}^{N-1} (x_i^T Q x_i + u_i^T R u_i) \right)$$

$$(V_0(x_0) = \min(J))$$

---

start with final state:

$$V_N(x_N) = x_N^T Q x_N \quad \text{(no more input to choose)}.$$

at time k, input u_k creats:

$$J = x_k^T Q x_k + u_k^T R u_k.$$

next state: $x_{k+1} = Ax_k + Bu_k$.

then $V_{k+1}(x_{k+1})$ is next state optimal cost plus to have optimal policy, minimize immediate cost and optimal future cost:

$$V_k(x) = \min_u \left[x_k^T Q x_k + u_k^T R u_k + V_{k+1}(x_{k+1}) \right]$$

$$= \min_u \left[x_k^T Q x_k + u_k^T R u_k + (Ax_k + Bu_k)^T \cdot P_{k+1} (Ax_k + Bu_k) \right]$$

→ Symmetry

since future cost to go = $V_{k+1}(x_{k+1}) = x_{k+1}^T P_{k+1} x_{k+1}$

$x_{k+1} = Ax_k + Bu_k$

so $V_{k+1}(x_{k+1}) = (Ax_k + Bu_k)^T P_{k+1} (Ax_k + Bu_k)$

$V_k(x) = \min_u \left(x^T Q x + u^T R u + (Ax+Bu)^T P_{k+1} (Ax+Bu) \right)$

$= x^T Q x + \min_u \left(u^T R u + (Ax)^T + (Bu)^T P_{k+1} (Ax + Bu) \right)$

$= x^T Q x + \min_u \left(u^T R u + (x^T A^T + u^T B^T) P_{k+1} (Ax + Bu) \right)$

$= x^T Q x + \min_u \left(u^T R u + (x^T A^T P_{k+1} + u^T B^T P_{k+1}) (Ax + Bu) \right)$

scalars (dot products)

$= x^T Q x + \min_u \left(u^T R u + \left(\underline{x^T A^T P_{k+1} \cdot Ax} + \boxed{x^T A^T P_{k+1} \cdot Bu} + \boxed{u^T B^T P_{k+1} \cdot Ax} + u^T B^T P_{k+1} \cdot Bu \right) \right)$

equal eo.

so $(x^T A^T P_{k+1} Bu)^T = (u^T B^T P_{k+1} Ax)$

$= (x^T Q x + x^T A^T P_{k+1} Ax) + \min_u \left(u^T R u + x^T A^T P_{k+1} Bu + u^T B^T P_{k+1} Ax + u^T B^T P_{k+1} Bu \right)$

equall

$= x^T (Q + A^T P_{k+1} A) x + \min_u \left(u^T (R + B^T P_{k+1} B) u + B^T P_{k+1} Ax + B^T P_{k+1} Bu \right) u$

$+ x^T A^T P_{k+1} Bu + 2 x^T A^T P_{k+1} Bu.$

$= x^T (Q + A^T P_{k+1} A) x + \min_u \left(u^T (R + B^T P_{k+1} Ax + B^T P_{k+1} B) u + x^T A^T P_{k+1} Bu \right)$

differentiate with respect to u: $2(R + B^T P_{k+1} Ax + B^T P_{k+1} B) u + 2 x^T A^T P_{k+1} B$

$= 0$

then $u = -(R+B^T P_{k+1} B)^{-1} \cdot B^T P_{k+1} A x$

$u_k^* = -K_k x_k$

since $V_k(x) = \min_{u_k} \left(x_k^T (Q + A^T P_{k+1} A) x_k + 2 u_k^T B^T P_{k+1} A x_k + u_k^T (R + B^T P_{k+1} B) u_k \right)$.

$u_k^T = -x_k^T K_k^T$

$\left(-2 x_k^T K_k^T B^T P_{k+1} A x_k + (-x_k^T K_k^T) \cdot (R + B^T P_{k+1} B)(-K_k x_k) \right)$

$= \left(-2 x_k^T K_k^T B^T P_{k+1} A x_k + (x_k^T K_k^T)(R + B^T P_{k+1} B)(K_k x_k) \right)$

So $\min_u V_k(x) = x_k^T (Q + A^T P_{k+1} A) x_k$

$-2 x_k^T K_k^T B^T P_{k+1} A x_k$

$+ (x_k^T K_k^T)(R + B^T P_{k+1} B)(K_k x_k)$

$= x_k^T \left(Q + A^T P_{k+1} A - 2 K_k^T B^T P_{k+1} A + K_k^T (R + B^T P_{k+1} B) K_k \right) x_k$.

$K_k^T \cdot (R + B^T P_{k+1} B) K_k$

$= R + B^T K_k^T \cdot (B^T P_{k+1} A x_k)$

symmetric.

$$K_k = (R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A$$

$$K_k^T = A^T \cdot P_{k+1} B \left(R + B^T P_{k+1} B\right)^{-1}$$

so $-2K_k^T B^T P_{k+1} A$

$$= -2 A^T \cdot P_{k+1} B \cdot \left(R + B^T P_{k+1} B\right)^{-1} \cdot \left(B^T P_{k+1} A\right)$$

$$\text{so } \min V(x) = x_k^T \Big(Q + A^T P_{k+1} A - 2 A^T P_{k+1} B \left(R + B^T P_{k+1} B\right)^{-1} \cdot \left(B^T P_{k+1} A\right) + K_k^T \left(B^T P_{k+1} A\right) \Big) x_k$$

also By definition $V_k(x_k) = x_k^T P_k x_k$

$$\text{so} \begin{cases} P_k = \left(Q + A^T P_{k+1} A - 2 A^T P_{k+1} B \left(R + B^T P_{k+1} B\right)^{-1} \cdot \left(B^T P_{k+1} A\right) + K_k^T \left(B^T P_{k+1} A\right) \right). \\ u_k^* = -K_k \cdot x_k. \end{cases}$$

$P_N = Q_N$ (final time $J = x_N^T Q_N x_N$)

LQR Workflow:

choose: A, B, Q, R, Q_N, N.

calculate: scare with $P_N = Q_N$ → P_{N-1} recursion

↓

RICCATI K_{N-1}

↓

repeat ← u_{k-1}

actuators ← $k_0, k_1, \ldots k_{N-1}$.

MPC (Model Predictive Control).

1. Linear MPC

① $x_{i+1} = Ax_i + Bu_i$

same objecceive as LQR (regulate cost to zero):

② $J = x_N^T P x_N + \sum_{i=0}^{N-1} (x_i^T Q x_i + u_i^T R u_i)$

③. starts with x_0:

$$\begin{cases} x_1 = Ax_0 + Bu_0 \\ x_2 = Ax_1 + Bu_1 \end{cases} \Rightarrow x_2 = A(Ax_0 + Bu_0) + Bu_1$$

$$x_3 = A^3 x_0 + AAB u_0 + ABu_1 + Bu_2$$

$$\vdots$$

$$x_i = A^i x_0 + \sum_{j=0}^{i-1} A^{i-1-j} B u_j$$

④. let $X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}$, $U = \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{N-1} \end{bmatrix}$

let $A^{\#} = \begin{bmatrix} A \\ A^2 \\ A^3 \\ \vdots \\ A^N \end{bmatrix}$ $B^{\#} = \begin{bmatrix} B & 0 & \cdots & 0 \\ AB & B & \cdots & 0 \\ A^2B & AB & \ddots & \vdots \\ A^{N-1}B & A^{N-2}B & \cdots & B \end{bmatrix}$

then $X = A^{\#} x_0 + B^{\#} U$

⑤ let $\bar{Q} = diag(Q, Q, \cdots, Q, P)$, $\bar{R} = diag(R, R, \cdots, R)$.

$$J = x^T \bar{Q} x + u^T \bar{R} u.$$

$$J = x^T \bar{Q} x + u^T \bar{R} u \qquad A = \begin{bmatrix} A \\ \vdots \\ A^N \end{bmatrix}, \; B = \text{matrix.}$$

$$x = Ax_0 + Bu.$$

$$J = (Ax_0 + Bu)^T \bar{Q} (Ax_0 + Bu) + u^T \cdot \bar{R} u.$$

$$= \left((Ax_0)^T + (Bu)^T\right) \bar{Q} (Ax_0 + Bu) + u^T \bar{R} u$$

$$= (x_0^T A^T + u^T B^T) \bar{Q} (Ax_0 + Bu) + u^T \bar{R} u.$$

$$= (x_0^T A^T \bar{Q} + u^T B^T \bar{Q}) (Ax_0 + Bu) + u^T \bar{R} u$$

$$= x_0^T A^T \bar{Q} \cdot Ax_0 + \underbrace{u^T B^T \bar{Q} Ax_0 + x_0^T A^T \bar{Q} Bu}_{\text{same scalar transpose}} + u^T B^T \bar{Q} Bu + u^T \bar{R} u.$$

$$J = x_0^T A^T \bar{Q} Ax_0 + 2u^T B^T \bar{Q} Ax_0 + u^T B^T \bar{Q} Bu + u^T \bar{R} u.$$

↳ doesn't depend on input command.

so to minimize $J \rightarrow \min_u \left(u^T (B^T \bar{Q} B + \bar{R}) u + 2u^T B^T \bar{Q} Ax_0 \right)$

quadratic program common form:

$$\min_u \left(\frac{1}{2} u^T H u + (g(x_0))^T \cdot u \right)$$

$$\begin{cases} H = B^T \bar{Q} B + \bar{R} \\ g(x_0) = 2B^T \bar{Q} A x_0 \end{cases}$$

let

$$\frac{\partial J}{\partial u} = Hu + (g(x_0))^T = 0 \quad \text{(without constraint).}$$

$$\boxed{u = H^{-1} \cdot g(x_0)} \rightarrow \text{same as LQR.}$$

add constraints : $\begin{cases} U_{min} \le U_i \le U_{max} \\ X_{min} \le X_i \le X_{max} \end{cases}$

$$\Downarrow$$

$$\begin{cases} U_{min} \le U \le U_{max} \\ X_{min} \le X_0 \le X_{max}. \\ X = Ax_0 + BU. \end{cases} \Rightarrow \begin{cases} X_{min} - Ax_0 \le BU \le X_{max} - Ax_0 \end{cases}$$

let $G = \begin{bmatrix} I \\ B \end{bmatrix}$ I: identity matrix.

$$l(x_0) = \begin{bmatrix} U_{min} \\ X_{min} - Ax_0 \end{bmatrix}, \quad u(x_0) = \begin{bmatrix} U_{max} \\ X_{max} - Ax_0 \end{bmatrix}.$$

final MPC Problem :

$$\begin{cases} \min_U \left(\frac{1}{2} U^T H U + g(x_0)^T U \right) \\ \text{subject to} \quad l(x_0) \le GU \le u(x_0) \end{cases} \quad \text{(Linear MPC)}.$$

---

① $x_{k+1} = f(x_k, u_k)$.

② at x_t, predice x_{k+1} using $x_{k+1} = f(x_k, u_k)$.

solve: $\min_{u_{0:N-1}} \left[\sum_{k=0}^{N-1} \left[(x_k^T Q x_k + u_k^T R u_k) + x_N^T Q_N x_N \right] \right.$

subject to $\begin{cases} x_{k+1} = f(x_k, u_k) \\ u_k \in [u_{min}, u_{max}] \\ q_{min}^k \in [q_{min}, q_{max}]. \end{cases}$

Machine-Learning.

Goal (min): $\hat{R}(\theta) = \frac{1}{n}\sum_{i=1}^{n} L(f_\theta(x_i), y_i)$ → model

- L → loss
- $f_\theta(x_i)$: x_i → input data
- y_i → correct data.
- θ → all adjustable model parameters.

gradient descend:

$L(\theta) = \frac{1}{2}(\hat{y} - y)^2$ (common square error).

$\nabla_\theta L = \begin{bmatrix} \frac{\partial L}{\partial \theta_1} \\ \frac{\partial L}{\partial \theta_2} \\ \vdots \\ \frac{\partial L}{\partial \theta_N} \end{bmatrix}$ → $\theta_{new} = \theta_{old} - \alpha \nabla_\theta L$

α = learning rate.

in ML: $\theta_{k+1} = \theta_k - \alpha \nabla_\theta \hat{R}(\theta_k)$

smoothness constant: S → $\| \nabla L(y) - \nabla L(x) \|$

$\| \nabla L(\theta_a) - \nabla L(\theta_b) \| \le S \| L(\theta_a - \theta_b) \|$

$L(\theta_a) \le L(\theta_b) + \nabla L(\theta_b)^T (a-b) + \frac{S}{2}\|a-b\|^2$ only for L-smooth !!

substitute $\theta_{i+1} = \theta_i - \alpha \nabla_\theta \hat{R}(\theta_k)$

→ $\hat{R}(\theta_{i+1}) \le \hat{R}(\theta_i) + \nabla R(\theta_i)^T \left(-\alpha \nabla_\theta \hat{R}(\theta_k)\right) + \frac{S}{2}\| -\alpha \nabla_\theta \hat{R}(\theta_k)\|^2$

↳ $\hat{R}(\theta_{i+1}) \le \hat{R}(\theta_i) - \alpha \left(\| \nabla R(\theta_i)\|^2\right) + \frac{S}{2}\|\alpha \nabla_\theta \hat{R}(\theta_k)\|^2$

choose $\alpha = \frac{1}{S}$ (usual conservative choice)

↳ $\hat{R}(\theta_{i+1}) \le \hat{R}(\theta_i) + \left(-\alpha + \frac{S\alpha^2}{2}\right) \cdot \|\nabla R(\theta_i)\|^2$

↳ $\hat{R}(\theta_{i+1}) \le \hat{R}(\theta_i) - \alpha\left(1 - \frac{S\alpha}{2}\right)\|\nabla R(\theta_i)\|^2$

$\frac{S\alpha}{2} < 1$

$0 < \alpha < \frac{2}{S}$

↳ $\hat{R}(\theta_{i+1}) \le \hat{R}(\theta_i) - \frac{1}{2S}\|\nabla R_\theta(\theta_i)\|^2$ → $\alpha\left(1 - \frac{S\alpha}{2}\right) > 0$

$1 - \frac{S\alpha}{2} > 0$

Proof : $d = y - x \rightarrow y = x + d \rightarrow$ let $x(t) = x + td$.

$\phi(t) = f(x+td) \rightarrow \phi'(t) = \nabla f(x+td)^T \cdot d$. $\quad t \in [0,1]$.

$f(y) - f(x) = \phi(1) - \phi(0) = \int_0^1 \nabla f(x+td)^T d \, dt$.

$f(y) - f(x) = \nabla f(x)^T \cdot d + \int_0^1 (\nabla f(x+td)^T - \nabla f(x)^T) d \cdot dt$.

$a^T b \leq \|a\| \cdot \|b\|$ ↗ ✓

$f(y) - f(x) \leq \nabla f(x)^T \cdot d + \int_0^1 \| \nabla f(x+td) - \nabla f(x) \| \, d \, dt$.

also $\|\nabla f(y) - \nabla f(x)\| \leq s \|y - x\|$ (L-smoothness)
I put s as smoothness constant

↓

$y - x = d$.

so $\|\nabla f(y) - \nabla f(x)\| \leq s \|d\|$

or let $\| \nabla f(x+td) - \nabla f(x) \| \leq s \cdot \|td\| \rightarrow st \|d\|$.

so $f(y) - f(x) \leq \nabla f(x)^T \cdot d + \int_0^1 st \|d\|^2 dt$. $\left(\int_0^1 t \, dt = \frac{1}{2} t^2 \Big|_0^1 = \frac{1}{2} \right)$

$f(y) - f(x) \leq \nabla f(x)^T \cdot d + \frac{1}{2} s \|d\|^2$

so $f(y) - f(x) \leq \nabla f(x)^T \cdot (y - x) + \frac{1}{2} \cdot s \| y - x \|^2$.

ural Network

$z = w^T x + b$, $a = \sigma(z)$

↓ linear → activation (non-Linear).

L: scalar scalar loss

k: Layer count

l: index of layer.

$$z^{(l)} = w^{T(l)} \cdot a^{(l-1)} + b^{(l)}$$
$$a^{(l)} = \sigma^{(l)}(z^{(l)})$$

Back propogation : layer error $\delta^{(l)} = \frac{\partial L}{\partial z^{(l)}}$

scalar loss: $L = l(a^{(k)}, y)$

layer error: $\delta^{(l)} = \frac{\partial L}{\partial z^{(l)}}$

at final layer: $a^k = \sigma^k(z^{(k)})$.

$\delta^{(k)} = \frac{\partial L}{\partial z^{(k)}} = \frac{\partial L}{\partial a^k} \odot \frac{\partial a^k}{\partial z^k}$. $\odot$ (element by element multiplication)

$\frac{\partial a^k}{\partial z^k} = \sigma^{(k)'}(z^k)$ (only respect to current layer).

$\delta^{(k)} = \nabla_{a^{(k)}} L \odot \sigma^{(k)'}(z^k)$

$$z_{k+1} = w^{T(k+1)} \cdot a^k + b^k$$

since $z_k \to a_k \to z_{k+1} \to L$.

$\delta^{(l)} = \frac{\partial L}{\partial z_k} = \frac{\partial a^k}{\partial z_k} \cdot \frac{\partial z_{k+1}}{\partial a_k} \cdot \frac{\partial L}{\partial z_{k+1}} \cdot \frac{\partial z_{k+1}}{\partial a_k} \cdot \frac{\partial a_k}{\partial z_k}$.

$\frac{\partial L}{\partial z_{k+1}} = \delta^{(k+1)}$

$\frac{\partial z_{k+1}}{\partial a_k} = w^{T(k+1)}$

$\frac{\partial a_k}{\partial z_k} = \sigma^{(k)'}(z^k)$.

$$w_{new} = w_{old} - \alpha \frac{\partial L}{\partial w^{(old)}}$$
$$b_{(new)} = b_{old} - \alpha \frac{\partial L}{\partial b^{(old)}}$$

So $\delta^{(l)} = (w^{T(l+1)} \cdot \delta^{(l+1)}) \odot \sigma^{(l)'}(z^l) = \frac{\partial L}{\partial z_l}$

$\because z_l = w^{T(l)} a^{(l-1)} + b^{(l)}$

$\therefore \frac{\partial L}{\partial w^{(l)}} = \frac{\partial L}{\partial z_l} \cdot \frac{\partial z_l}{\partial w^{(l)}} = \delta^{(l)} \cdot (a^{(l-1)})^T$; $\frac{\partial L}{\partial b^{(l)}} = \frac{\partial L}{\partial z_l} \cdot \frac{\partial z_l}{\partial b^{(l)}} = \delta^{(l)} \cdot 1 = \delta^{(l)}$.

Reinforcement Learning.

state at time t: $s_t \in S$ (state space).

action at time t from controller: $a_t \in A$ (action space).

Policy $\pi(a_t|s_t)$ → given state s, what probability should be assigned to action a?

$a_t \sim \pi(\cdot|s_t)$ → action at t is sampled from policy's probability distribution.

covariance $(X, Y) = \frac{\sum_{i=1}^{N}(x_i - \bar{x})(y_i - \bar{y})}{N-1}$ → $\sigma(s_t) = \sqrt{\frac{\Sigma (s_t - \bar{s_t})^2}{N-1}}$ → covariance $(s_t, s_t) = (\sigma(s_t))^2$

1 dim

Gaussian Policy: stochastic policy that outputs a probability distributions over possible actions

$\pi(a_t|s_t) = N(\mu_\theta(s_t), \Sigma_\theta(s_t))$.

N: Gaussian distribution. $\Sigma_\theta(s_t)$: covariance of action

$\mu_\theta(s_t)$: mean action under given state

θ: neural network parameters.

$$N(\mu_\theta(s_t), \Sigma_\theta(s_t)) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{\left(-\frac{(a_t - \mu)^2}{2\sigma^2}\right)}$$

σ: standard deviation of actions.

So in order:

$s_t \rightarrow \begin{cases} \mu_\theta(s_t) \text{ (mean action)} \\ \Sigma_\theta(s_t) \text{ (covariance of action)} \rightarrow \sigma(s_t) \text{ (std)} \end{cases} \rightarrow N(\mu_\theta(s_t), \Sigma_\theta(s_t)) \rightarrow a_t \text{ (chosen action)}$

another same work flow:

L_θ is a matrix where $\Sigma_\theta = L_\theta L_\theta^T$

$a_t = \mu_\theta(s_t) + \sigma_\theta(s_t)\,\epsilon_t$, $\epsilon_t \sim N(0, 1)$

after action: $a_t = \mu_\theta(s_t) + \text{noise}$ (e.g. $L_\theta(s_t)\,\varepsilon_t$).

then move to next state in the new environment:

$$s_{t+1} \sim P(s_{t+1} \mid s_t, a_t)$$

P (transition law): environment physics.

then evaluate reward: $r_t = r(s_t, a_t, s_{t+1})$.

stores: $(s_t, a_t, r_t, s_{t+1}, d_t)$.

PPO also stores: $\left(\log \pi_{\theta_{old}}(a_t \mid s_t)\right)$.

$$\left(s_t, a_t, r_t, s_{t+1}, d_t, \log \pi_{\theta_{old}}(a_t \mid s_t), V_\phi(s_t)\right).$$

return G_t: $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$. $= r_{t0} + \gamma \cdot r_{t+1} + \gamma^2 r_{t+2} + \cdots$ (total discounted future reward).

objective is to also max $J(\theta) = E_{\tau_\theta}[G_t]$. (probability weighted average of G_t).

trajectory: $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_t)$

starting state probability

$$P_\theta(\tau) = p(s_t) \prod_{k=t}^{T-1} \pi_\theta(a_k \mid s_k) \cdot P(s_{k+1} \mid s_k, a_k).$$

$\prod_{k=t}^{T-1}$: multiply all from $k=t$ to $T-1$.

$$E_{\tau \sim P_\theta(\tau)}[G_t] = \sum_{\tau} P_\theta(\tau) \cdot G_t(\tau).$$

if starting state is known:

state-value function: $V^\pi(s_t) = E_\pi[G_t \mid s_t = s_t]$ (starting from state s_t, what future return should we expect if we continue following policy π?)

$$P_\theta(\tau \mid s_t) = \prod_{k=t}^{T-1} \pi_\theta(a_k \mid s_k)\, P(s_{k+1} \mid s_k, a_k).$$

action-value function:

$$Q^\pi(s_t, a_t) = E_\pi[G_t \mid s_t = s_t, A_t = a_t].$$

(if taken action in state s_t, continue follow policy, what return should we expect?)

$V^\pi(s_t)$ = expected return before choosing a particular action.

$Q^\pi(s_t, a_t)$ = expected return after fixing current action.

Advantage function:

$$A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$$

(how much more than expected return this action gives?)

$V_\phi(s_t) \approx V^\pi(s_t)$

$$V^\pi(s_t) = E_\pi\left[r_t + \gamma V^\pi(s_{t+1}) \mid s_t \right].$$

Complete PPO algorithm:

1. initialize actor & critics: $\pi_\theta(a_t|s_t) = \mathcal{N}(\mu_\theta(s_t), \Sigma_\theta(s_t))$
2. observe s_t.
3. sample a_t from $\pi_\theta(a_t|s_t)$., then evaluate new s_{t+1} under $P(s_{t+1}|s_t a_t)$.
4. calculate reward: $r_t = r(s_t, a_t, s_{t+1})$

   stores: $(s_t, a_t, r_t, s_{t+1}, d_t)$, $V_{\phi_{old}}(s_t)$, $\log \pi_{\theta_{old}}(a_t|s_t)$.

   if $d_t = 1$, terminates. $d_t = 0 \rightarrow$ continue.
5. repeat 2-4 N steps. stores $\tau = (s_0, a_0, r_0, s_1, a_1, r_1 \cdots)$.
6. calculate TD Errors: $\delta_t = r_t + \gamma(1 - d_t) V_{\phi_{old}}(s_{t+1}) - V_{\phi_{old}}(s_t)$
7. calculate advantage: $\hat{A}_t = \delta_t + \gamma\lambda(1 - d_t)\hat{A}_{t+1}$

   ↓ normalize

   $$\vec{A}_t = \frac{\vec{A}_t - \text{mean}(\vec{A})}{\text{std}(\vec{A}) + \varepsilon_{num}}$$
8. construct critic targets:

   $$\vec{V}_t = \vec{A}_t + V_{\phi_{old}}(s_t)$$

   critic loss: $\nabla_\phi L(\phi) = \frac{1}{N} \sum_t^N \left[V_\phi(s_t) - \hat{V}_t \right]^2$

9. calculate PPO Probability ratio.

$$P_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} = e^{[\log_e \pi_\theta(a_t|s_t) - \log_e \pi_{\theta_{old}}(a_t|s_t)]}$$

10. calculate clipped Actor objective:

$$L_{clip}(\theta) = \frac{1}{N} \sum_t \min\left[P_t(\theta)\hat{A}_t, \text{clip}(P_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t\right].$$

$\varepsilon = 0.2$ (common).

11. update both networks.

$$L_{total} = -L_{clip} + c_1 L_V - c_2 H(\pi_\theta)$$

- $-L_{clip}$: actor loss
- $c_1 L_V$: critic loss
- $c_2 H(\pi_\theta)$: policy entropy. encourages exploration.

back propogation calculate ∇L.

then update θ, ϕ by

$$\begin{cases} \theta_{new} = \theta_{old} - \alpha_\theta \nabla_\theta L \\ \phi_{new} = \phi_{old} - \alpha_\phi \nabla_\phi L \end{cases}$$

12. Repeat. deploy $a_t = \mu_\theta(s_t)$.